팀4 — AGV 자재공급 · SLA 미준수 위험 예측

K-뉴딜 아카데미 스마트팩토리 캡스톤 · 3~6회차 통합 실습

데이터	분석단위	규모	기간
`team4_agv.csv`	자재배송 요청 1건	1,600행 × 26열	2026년 7월 1일 ~ 7월 23일 (AGV A01~A06, 경로 R1~R5)

이 노트북에서 하는 일

데이터 탐색 → 시각화 → 전처리 → 머신러닝 모델 3종 → 성능평가 → 특성중요도

현장 상황

가상의 **전자부품공장**입니다. **AGV(무인운반차)** 가 창고와 조립라인 사이에서 자재를 운반합니다. 자재요청이 늘면서 일부 배송이 늦어지고, 심한 경우 조립라인의 자재가 떨어져 **생산이 중단(라인스톱)** 됩니다.

여기서 **SLA(Service Level Agreement)** 개념이 중요합니다. 이 공장은 요청 우선순위에 따라 **목표 배송시간**을 약속해 두었습니다.

우선순위	목표시간(SLA)	건수
URGENT	8분	165건
HIGH	12분	420건
NORMAL	18분	1,015건

즉 **같은 10분 배송이 URGENT에서는 위반이고 NORMAL에서는 준수**입니다. "늦었다"는 판정은 절대 시간이 아니라 **약속 대비 상대적 판정**입니다.

물류 담당자는 요청이 들어오는 순간 **배차 순서, 우회경로, 충전 시점**을 결정해야 합니다.

핵심 질문

> **SLA 미준수와 라인스톱을 줄이려면, 요청 접수 시점에 어떤 요청이 위험한지 알 수 있는가? 배차·우회·충전 운영을 어떻게 바꿔야 하는가?**

판단 시점

> **자재요청이 접수된 순간 (우선순위·거리·경로·혼잡도·배터리는 알고, 실제 이동시간과 배송결과는 모름)**

판단 시점이 중요한 이유: **그 시점에 알 수 없는 값**을 입력으로 쓰면 시험 답안지를 미리 본 것과 같아집니다(**데이터 누출**). 아래 8절에서 직접 걸러냅니다.

---

1. 준비 — 라이브러리와 한글 폰트

도구	역할
`pandas` (pd)	표 데이터 처리 (엑셀에 해당)
`numpy` (np)	숫자 계산
`matplotlib` (plt) · `seaborn` (sns)	그래프
`scikit-learn`	머신러닝

아래 셀은 **한 번만** 실행하면 됩니다.

```python
# [준비 셀] 내용을 자세히 이해할 필요는 없습니다. 실행만 하세요.
# 라이브러리 불러오기 + 한글 폰트 설정
import os, sys, glob, warnings
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report, roc_curve,
                             roc_auc_score, recall_score, precision_score)
warnings.filterwarnings("ignore")
IN_COLAB = "google.colab" in sys.modules

# ------------------------------------------------------------------------
# 한글 폰트 설정
#   ※ 순서 주의: seaborn 스타일을 먼저 적용해야 한다.
#      sns.set_style() 이 글꼴 설정을 sans-serif 로 덮어쓰기 때문이다.
# ------------------------------------------------------------------------
sns.set_style("whitegrid")                      # ① 스타일 먼저

from matplotlib import font_manager as fm

KOREAN_FONTS = ["NanumGothic", "NanumBarunGothic", "Malgun Gothic", "AppleGothic",
                "Apple SD Gothic Neo", "Noto Sans CJK KR", "Noto Sans KR"]


def find_korean_font():
    """설치된 한글 폰트를 찾고, 없으면 Colab에서 나눔폰트를 설치한다."""
    have = {f.name for f in fm.fontManager.ttflist}
    for n in KOREAN_FONTS:
        if n in have:
            return n
    if IN_COLAB:                                # 나눔폰트 설치 후 다시 찾기
        print("한글 폰트를 설치하는 중입니다... (20초 정도 걸립니다)")
        os.system("apt-get install -y fonts-nanum > /dev/null 2>&1")
        os.system(f"{sys.executable} -m pip install -q koreanize-matplotlib")
        for path in glob.glob("/usr/share/fonts/truetype/nanum/Nanum*.ttf"):
            fm.fontManager.addfont(path)
        have = {f.name for f in fm.fontManager.ttflist}
        for n in KOREAN_FONTS:
            if n in have:
                return n
    return None

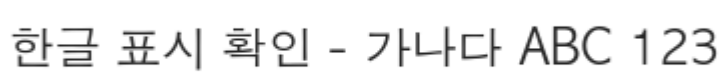

FONT = find_korean_font()                       # ② 폰트 나중에
if FONT:
    plt.rcParams["font.family"] = FONT
    plt.rcParams["font.sans-serif"] = [FONT] + list(plt.rcParams["font.sans-serif"])
plt.rcParams["axes.unicode_minus"] = False      # 음수 기호(-) 깨짐 방지
plt.rcParams["figure.figsize"] = (9, 4.5)
plt.rcParams["figure.dpi"] = 110

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

print("준비 완료 | pandas", pd.__version__, "| Colab:", IN_COLAB)
print("한글 폰트 :", FONT if FONT else "찾지 못함")
```

```
# ③ 자가진단 — 아래 그림에 한글이 보이면 정상입니다
plt.figure(figsize=(5, 0.9))
plt.text(0.5, 0.5, "한글 표시 확인 — 가나다 ABC 123",
         ha="center", va="center", fontsize=15)
plt.axis("off")
plt.show()
print("위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.")
```

```
준비 완료 | pandas 2.2.3 | Colab: False
한글 폰트 : AppleGothic
```




```
위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.
```

---

2. 데이터 가져오기

두 가지 방법이 있습니다. **① GitHub에서 바로 내려받기**(권장)가 안 되면 **② PC에서 업로드**를 쓰세요.

```
# ① GitHub에서 바로 내려받기
URL = "https://raw.githubusercontent.com/StillWork/capstone2610/main/team4_agv.csv"
df_raw = pd.read_csv(URL)

# ② PC에서 업로드 — ①이 안 되면 위 2줄을 지우고 아래 3줄의 주석을 푸세요
# from google.colab import files
# up = files.upload()
# df_raw = pd.read_csv(list(up.keys())[0], encoding="utf-8-sig")

print("크기(행, 열):", df_raw.shape)
df_raw.head(3)
```

```
크기(행, 열): (1600, 26)
```

	record_id	request_time	shift	request_zone	destination_line	material_type	priority_level	agv_id	route
0	AG-00001	2026-07-01 00:00:00	Day	Z2	L2	MAT_D	NORMAL	A02	
1	AG-00002	2026-07-01 00:20:00	Night	Z3	L4	MAT_D	URGENT	A05	
2	AG-00003	2026-07-01 00:40:00	Day	Z2	L2	MAT_A	HIGH	A02	

---

3. 데이터 훑어보기

새 데이터를 받으면 **크기 → 자료형·결측 → 요약통계** 순으로 봅니다. 요약통계에서는 **불가능한 값**(음수 수량, 범위를 벗어난 값)이 있는지 확인합니다.

```
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1600 entries, 0 to 1599
Data columns (total 26 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   record_id                     1600 non-null   object 
 1   request_time                  1600 non-null   object 
 2   shift                         1600 non-null   object 
 3   request_zone                  1600 non-null   object 
 4   destination_line              1600 non-null   object 
 5   material_type                 1600 non-null   object 
 6   priority_level                1600 non-null   object 
 7   agv_id                        1600 non-null   object 
 8   route_id                      1600 non-null   object 
 9   distance_m                    1600 non-null   int64  
 10  congestion_level              1600 non-null   object 
 11  battery_start_pct             1590 non-null   float64
 12  battery_end_pct               1600 non-null   float64
 13  loading_time_sec              1600 non-null   float64
 14  dispatch_wait_sec             1600 non-null   float64
 15  travel_time_sec               1586 non-null   float64
 16  unloading_time_sec            1600 non-null   float64
 17  obstacle_event_count          1600 non-null   int64  
 18  reroute_flag                  1600 non-null   object 
 19  charge_before_dispatch_flag   1600 non-null   object 
 20  sla_target_min                1600 non-null   int64  
 21  actual_delivery_min           1600 non-null   float64
 22  sla_met_flag                  1600 non-null   object 
 23  line_stock_at_request_min     1600 non-null   float64
 24  line_stop_flag                1600 non-null   object 
 25  line_stop_min                 1600 non-null   float64
dtypes: float64(9), int64(3), object(14)
memory usage: 325.1+ KB
```

```
display(df_raw.describe().T.round(2))

miss = df_raw.isna().sum()
print("[결측이 있는 변수]")
print(miss[miss > 0].to_string() if (miss > 0).any() else "없음")
```

	count	mean	std	min	25%	50%	75%	max
distance_m	1,600.000	343.000	123.910	90.000	241.000	335.000	447.000	608.000
battery_start_pct	1,590.000	61.720	21.630	12.000	47.420	61.650	78.070	100.000
battery_end_pct	1,600.000	50.740	20.290	5.000	35.800	49.600	66.100	95.300
loading_time_sec	1,600.000	67.790	16.990	25.000	56.000	66.700	78.550	132.000
dispatch_wait_sec	1,600.000	84.410	45.010	5.000	52.600	78.500	113.000	228.500
travel_time_sec	1,586.000	246.720	108.320	44.800	167.000	229.200	309.380	642.400
unloading_time_sec	1,600.000	49.850	11.820	20.000	41.780	49.600	57.800	86.900
obstacle_event_count	1,600.000	0.830	0.890	0.000	0.000	1.000	1.000	4.000
sla_target_min	1,600.000	15.390	3.600	8.000	12.000	18.000	18.000	18.000
actual_delivery_min	1,600.000	7.650	2.390	3.180	5.880	7.240	9.000	16.500
line_stock_at_request_min	1,600.000	20.850	8.760	1.000	15.200	21.600	27.100	44.500
line_stop_min	1,600.000	0.160	1.360	0.000	0.000	0.000	0.000	17.300

```
[결측이 있는 변수]
battery_start_pct    10
travel_time_sec      14
```

---

4. 데이터 정제

항목	처리
출발 배터리 결측 10건	**AGV별 중앙값**으로 채움
이동시간 결측 14건	**경로별 중앙 속도 × 거리**로 추정 (거리를 알고 있으므로 물리적 추정이 가능)
충전 66건의 배터리 증가	**정상 데이터** — 출발 전 충전을 반영한 값이므로 수정하지 않음

```python
df = df_raw.copy()

# ① 출발 배터리 결측 → AGV별 중앙값
n = int(df["battery_start_pct"].isna().sum())
df["battery_start_pct"] = df["battery_start_pct"].fillna(
    df.groupby("agv_id")["battery_start_pct"].transform("median"))
print(f"출발 배터리 결측 {n}건 → AGV별 중앙값으로 채움")

# ② 이동시간 결측 → 경로별 중앙 속도(m/s) × 거리 로 추정
n = int(df["travel_time_sec"].isna().sum())
speed = (df["distance_m"] / df["travel_time_sec"]).replace([np.inf, -np.inf], np.nan)
route_speed = speed.groupby(df["route_id"]).transform("median").fillna(speed.median())
df["travel_time_sec"] = df["travel_time_sec"].fillna(df["distance_m"] / route_speed)
print(f"이동시간 결측 {n}건 → 경로별 속도 × 거리로 추정")

df["request_time"] = pd.to_datetime(df["request_time"])
print("\n남은 결측:", int(df.isna().sum().sum()), "건")
```

```
출발 배터리 결측 10건 → AGV별 중앙값으로 채움
이동시간 결측 14건 → 경로별 속도 × 거리로 추정

남은 결측: 0 건
```

---

5. 목표변수 — 찾아내고 싶은 사건은 얼마나 드문가

목표변수는 **`sla_met_flag`** 입니다. SLA 미준수(N) 을 **1**, 나머지를 **0** 으로 둡니다.

양성이 드물면 **"전부 음성"이라고만 답해도 정확도가 매우 높습니다.** 그래서 이 데이터에서는 **정확도를 믿으면 안 됩니다.**

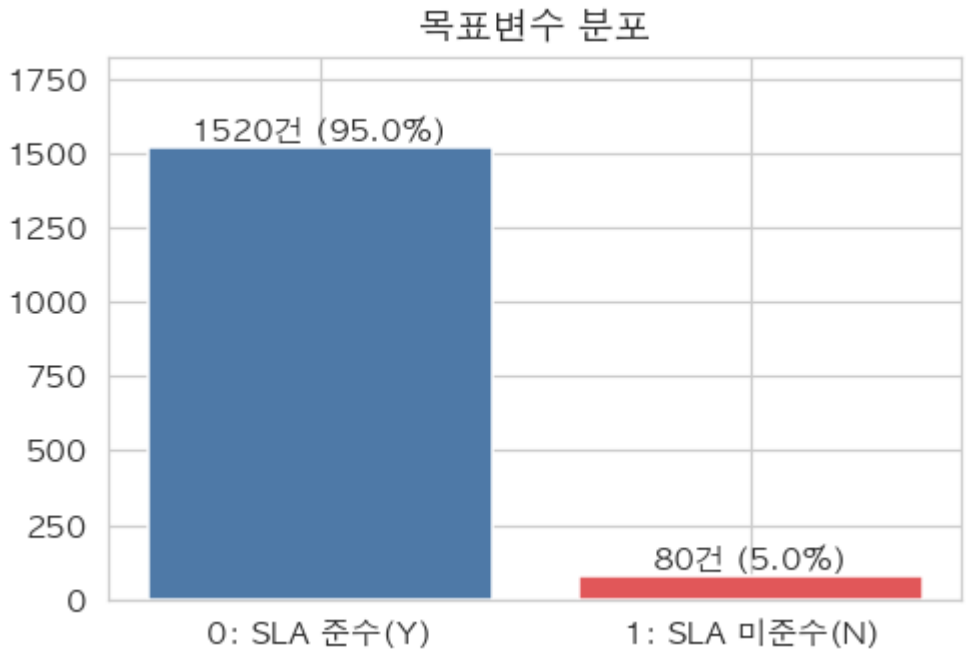
```python
y = (df['sla_met_flag'] == 'N').astype(int)          # 1 = SLA 미준수(N)
df["target"] = y

print(df["sla_met_flag"].value_counts().to_string())
print(f"\n양성(1) 비율            : {y.mean()*100:.2f}%")
print(f"'전부 음성' 모델의 정확도 : {(1-y.mean())*100:.2f}%  ← 이 숫자를 넘어야 의미가 있다")

cnt = y.value_counts().sort_index()
plt.figure(figsize=(5, 3.2))
plt.bar(["0: SLA 준수(Y)", "1: SLA 미준수(N)"], cnt.values, color=["#4e79a7", "#e15759"])
for i, v in enumerate(cnt.values):
    plt.text(i, v, f"{v}건 ({v/len(y)*100:.1f}%)", ha="center", va="bottom")
plt.ylim(0, cnt.values.max() * 1.2)
plt.title("목표변수 분포")
plt.show()
```

```
sla_met_flag
Y    1520
N      80

양성(1) 비율            : 5.00%
'전부 음성' 모델의 정확도 : 95.00%  ← 이 숫자를 넘어야 의미가 있다
```



---

6. 탐색 ① 그룹별 양성률 — 어느 조건에서 문제가 많은가

반드시 표본 수를 함께 보세요. "10건 중 2건(20%)"과 "400건 중 40건(10%)"은 비율만 보면 앞이 2배 높지만, 앞은 1건만 달라져도 크게 흔들립니다.

```python
def group_rate(col):
    g = df.groupby(col)["target"]
    t = pd.DataFrame({"전체건수": g.size(), "양성건수": g.sum().astype(int)})
    t["양성률(%)"] = (t["양성건수"] / t["전체건수"] * 100).round(2)
    return t


GROUPS = ["priority_level", "congestion_level", "route_id", "agv_id"]
for c in GROUPS:
    print(f"===== {c} =====")
    display(group_rate(c))
```

```
===== priority_level =====
```

priority_level	전체건수	양성건수	양성률(%)
HIGH	420	33	7.860
NORMAL	1015	0	0.000
URGENT	165	47	28.480

```
===== congestion_level =====
```

congestion_level	전체건수	양성건수	양성률(%)
HIGH	312	62	19.870
LOW	662	2	0.300
MID	626	16	2.560

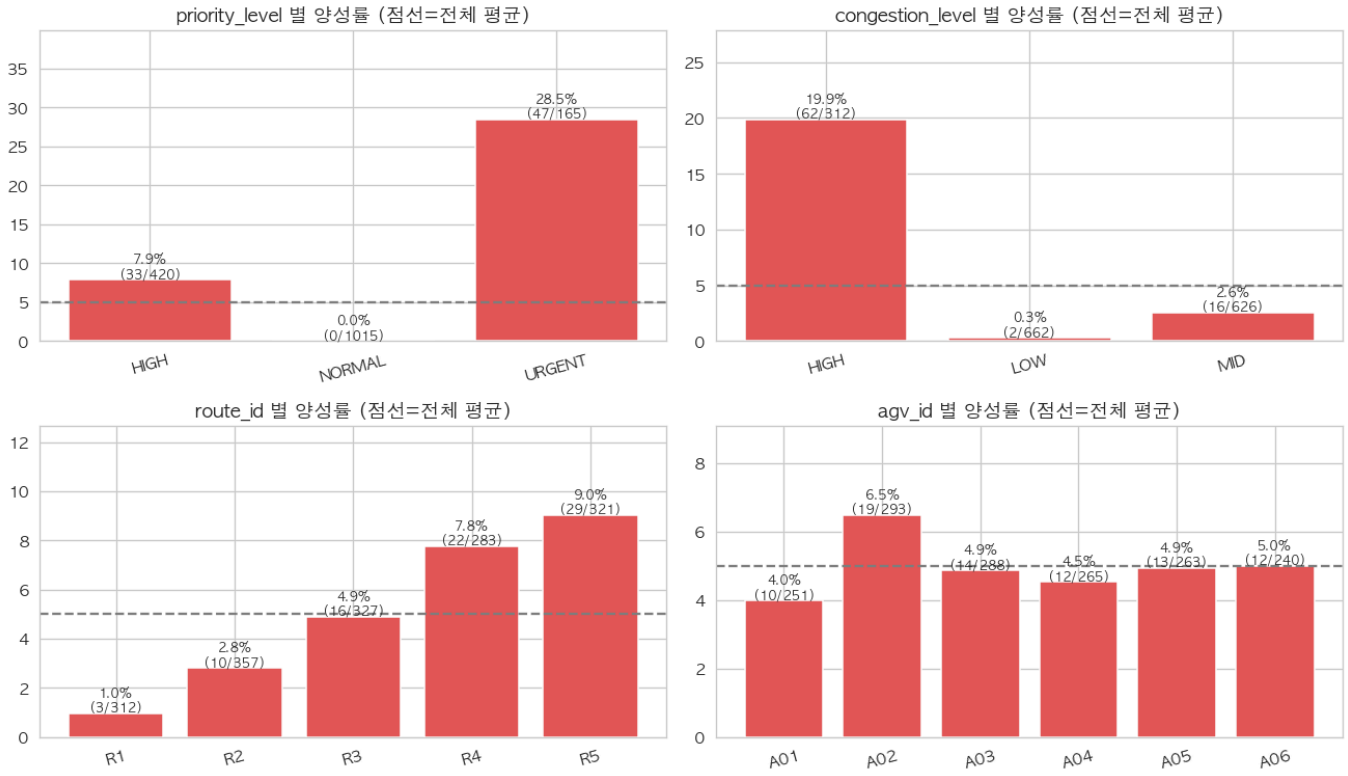
```
===== route_id =====
```

route_id	전체건수	양성건수	양성률(%)
R1	312	3	0.960
R2	357	10	2.800
R3	327	16	4.890
R4	283	22	7.770
R5	321	29	9.030

```
===== agv_id =====
```

agv_id	전체건수	양성건수	양성률(%)
A01	251	10	3.980
A02	293	19	6.480
A03	288	14	4.860
A04	265	12	4.530
A05	263	13	4.940
A06	240	12	5.000

```
overall = df["target"].mean() * 100
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for ax, c in zip(axes.ravel(), GROUPS):
    t = group_rate(c)
    ax.bar(t.index.astype(str), t["양성률(%)"], color="#e15759")
    ax.axhline(overall, color="gray", ls="--")
    for i, (n, m, r) in enumerate(zip(t["전체건수"], t["양성건수"], t["양성률(%)"])):
        ax.text(i, r, f"{r:.1f}%\n({m}/{n})", ha="center", va="bottom", fontsize=8)
    ax.set_title(f"{c} 별 양성률 (점선=전체 평균)")
    ax.set_ylim(0, max(t["양성률(%)"].max() * 1.4, 1))
    ax.tick_params(axis="x", rotation=15)
plt.tight_layout()
plt.show()
```



---

7. 탐색 ② 교차표 — 두 조건을 겹쳐서 보기

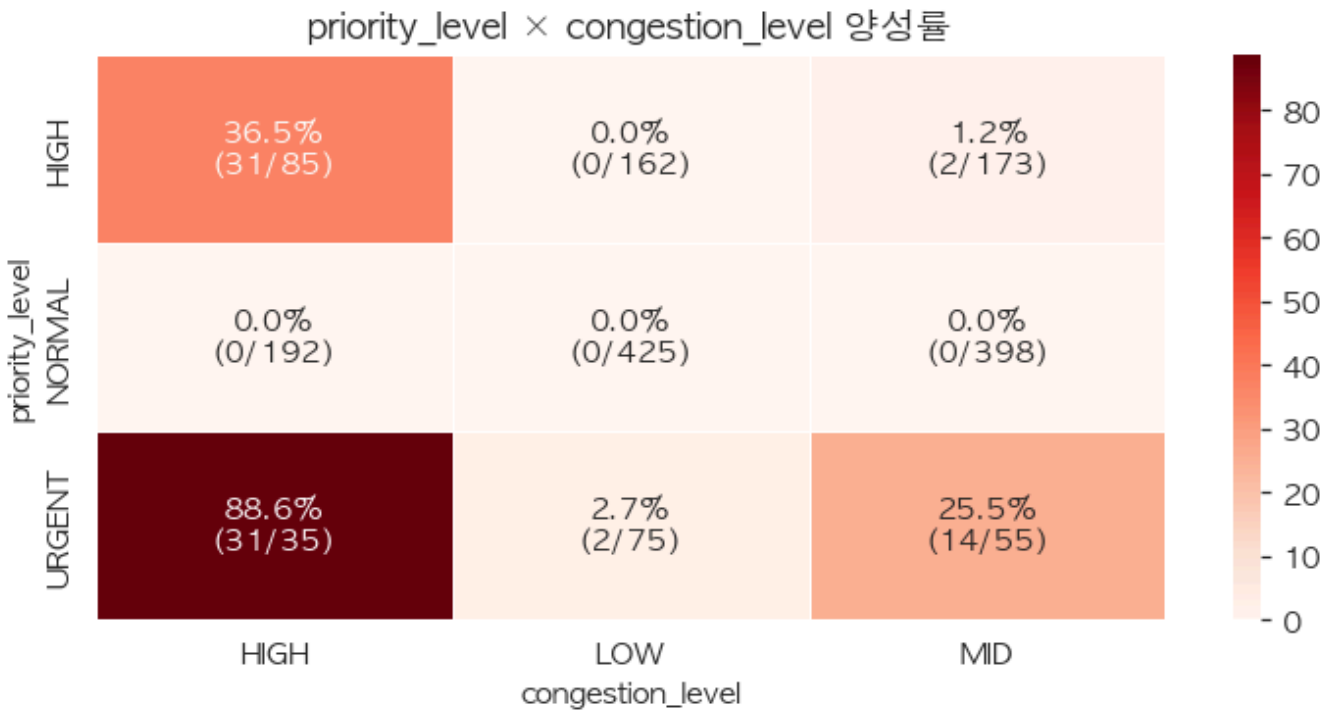
그룹을 하나씩 볼 때 보이는 차이가, 사실은 **다른 변수 때문**일 수 있습니다(**교란**). `priority_level` 와 `congestion_level` 를 겹쳐서 확인합니다.

```
GX, GY = "priority_level", "congestion_level"

cnt = pd.crosstab(df[GX], df[GY])
pos = pd.crosstab(df[GX], df[GY], values=df["target"], aggfunc="sum").fillna(0)
rate = (pos / cnt * 100).round(1)

plt.figure(figsize=(8, 3.4))
annot = rate.astype(str) + "%\n(" + pos.astype(int).astype(str) + "/" + cnt.astype(str) + ")"
sns.heatmap(rate, annot=annot, fmt="", cmap="Reds", linewidths=0.5)
plt.title(f"{GX} × {GY} 양성률")
plt.show()

print("칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.")
```



칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.

---

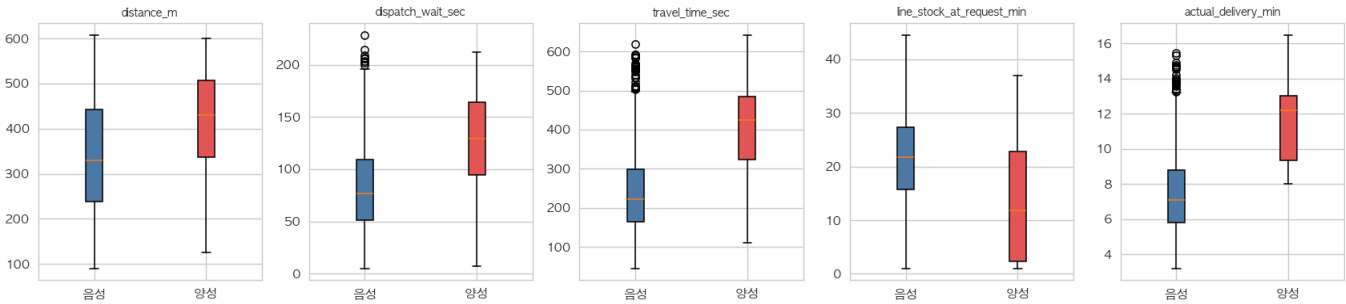
8. 탐색 ③ 수치형 변수 비교 — 양성/음성은 무엇이 다른가

상자가 많이 겹치면 **그 변수 하나로는 구분할 수 없다**는 뜻입니다. 제조 데이터는 대부분 이렇고, 그래서 **여러 변수를 조합하는 머신러닝**이 필요합니다.

```python
NUM_VIZ = [c for c in [
    "distance_m", "dispatch_wait_sec", "travel_time_sec", "line_stock_at_request_min",
    "actual_delivery_min"
] if c in df.columns]

fig, axes = plt.subplots(1, len(NUM_VIZ), figsize=(3.1 * len(NUM_VIZ), 3.6))
for ax, c in zip(np.atleast_1d(axes).ravel(), NUM_VIZ):
    data = [df.loc[df["target"] == 0, c].dropna(), df.loc[df["target"] == 1, c].dropna()]
    bp = ax.boxplot(data, patch_artist=True)
    for patch, color in zip(bp["boxes"], ["#4e79a7", "#e15759"]):
        patch.set_facecolor(color)
    ax.set_xticks([1, 2])
    ax.set_xticklabels(["음성", "양성"])
    ax.set_title(c, fontsize=9)
plt.tight_layout()
plt.show()

cmp = pd.DataFrame({"음성 평균": df[df["target"] == 0][NUM_VIZ].mean(),
                    "양성 평균": df[df["target"] == 1][NUM_VIZ].mean()})
cmp["차이"] = cmp["양성 평균"] - cmp["음성 평균"]
display(cmp.round(3))
```



	음성 평균	양성 평균	차이
distance_m	339.206	415.050	75.844
dispatch_wait_sec	82.221	125.910	43.689
travel_time_sec	238.118	407.194	169.076
line_stock_at_request_min	21.263	12.934	-8.329
actual_delivery_min	7.442	11.531	4.089

---

9. 탐색 ④ 산점도와 상관관계

산점도는 점 하나가 기록 하나입니다. 색으로 양성/음성을 구분하면 **문제가 어느 영역에 모여 있는지** 눈으로 확인할 수 있습니다.

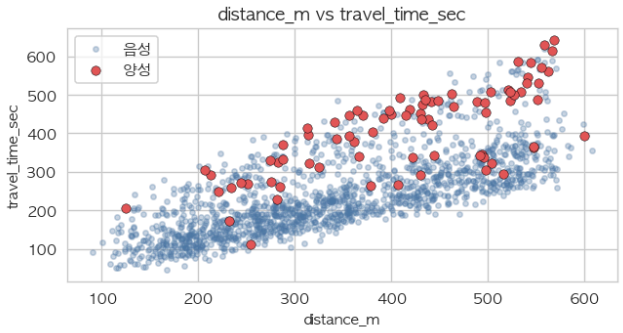
> 거리와 이동시간의 관계 - 기울기의 역수가 평균 속도, 직선에서 벗어난 점이 '막힌 운행'

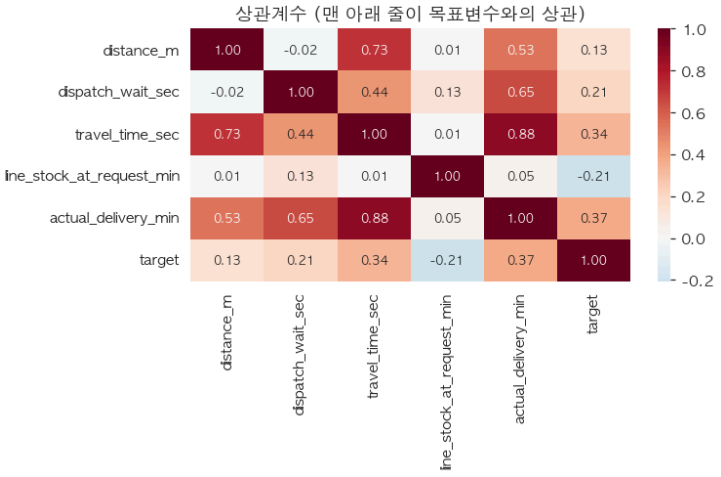
상관계수는 **직선 관계의 강도**(-1~+1)입니다. 세 가지를 기억하세요: 곡선 관계는 못 잡고, **상관은 인과가 아니며**, 이상치에 민감합니다.

```
X1, Y1 = "distance_m", "travel_time_sec"

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
neg, pos_ = df[df["target"] == 0], df[df["target"] == 1]
axes[0].scatter(neg[X1], neg[Y1], s=12, alpha=0.3, c="#4e79a7", label="음성")
axes[0].scatter(pos_[X1], pos_[Y1], s=38, c="#e15759", edgecolors="black",
                linewidths=0.3, label="양성")
axes[0].set_xlabel(X1)
axes[0].set_ylabel(Y1)
axes[0].set_title(f"{X1} vs {Y1}")
axes[0].legend()

corr = df[NUM_VIZ + ["target"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=axes[1],
            annot_kws={"size": 8})
axes[1].set_title("상관계수 (맨 아래 줄이 목표변수와의 상관)")
plt.tight_layout()
plt.show()
```





10. 입력변수 고르기 — 데이터 누출 막기

판단 시점(자재요청이 접수된 순간 (우선순위·거리·경로·혼잡도·배터리는 알고, 실제 이동시간과 배송결과는 모름))에 **이미 알 수 있는 변수만** 사용합니다.

쓸 수 없는 변수 (결과가 나온 뒤에 생기는 값)

변수	이유
`battery_end_pct`	도착 배터리. 배송이 끝나야 안다
`travel_time_sec`	이동시간. 배송이 끝나야 안다 (배송시간의 최대 구성요소)
`unloading_time_sec`	하역시간. 배송이 끝나야 안다
`actual_delivery_min`	실제 배송시간. SLA 판정의 근거값 → 가장 치명적인 누출
`line_stop_flag`	라인정지 여부. 사고가 난 뒤의 결과다
`line_stop_min`	라인정지 시간. 사고 이후 집계된다

> 성능이 비현실적으로 높으면(정확도 99%, AUC 0.99) **먼저 누출을 의심하세요.** 실무에서 가장 흔한 실패 원인입니다.

```
CAT = [c for c in [
    "shift", "request_zone", "destination_line", "material_type", "priority_level",
    "agv_id", "route_id", "congestion_level"
] if c in df.columns]      # 범주형 입력
NUM = [c for c in [
    "distance_m", "sla_target_min", "battery_start_pct", "line_stock_at_request_min"
] if c in df.columns]      # 수치형 입력
LEAK = [
    "battery_end_pct", "travel_time_sec", "unloading_time_sec", "actual_delivery_min",
    "line_stop_flag", "line_stop_min", "record_id"
]                                  # 사용 금지(데이터 누출)

print(f"입력 변수: 범주형 {len(CAT)}개 + 수치형 {len(NUM)}개 = {len(CAT)+len(NUM)}개")
print("범주형:", CAT)
print("수치형:", NUM)
```

```
입력 변수: 범주형 8개 + 수치형 4개 = 12개
범주형: ['shift', 'request_zone', 'destination_line', 'material_type', 'priority_level', 'agv_i
d', 'route_id', 'congestion_level']
수치형: ['distance_m', 'sla_target_min', 'battery_start_pct', 'line_stock_at_request_min']
```

11. 전처리 — 원핫인코딩 · 분리 · 스케일링

처리	이유
원핫인코딩	모델은 문자를 모른다. 범주마다 0/1 열을 만든다
학습/평가 분리	학습에 쓴 데이터로 평가하면 항상 성적이 좋다(시험문제를 미리 본 셈)
스케일링	변수마다 단위·범위가 달라서 큰 숫자 변수가 과대평가된다

순서가 중요합니다. 스케일러는 **학습 데이터에서만** 기준(평균·표준편차)을 배우고, 평가 데이터에는 그 기준을 **적용만** 해야 합니다. 전체 데이터로 먼저 스케일링하면 평가 데이터의 정보가 학습에 새어 들어갑니다(**데이터 누출**).

```
# ① 원핫인코딩 + 입력행렬 만들기
X = pd.concat([df[NUM], pd.get_dummies(df[CAT])], axis=1)
print("원핫인코딩 후 열 개수:", X.shape[1])

# ② 학습 75% / 평가 25% 로 분리 (stratify=y 로 양성 비율을 양쪽에 똑같이 맞춘다)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# ③ 스케일링 – 반드시 학습 데이터에서만 fit !
sc = StandardScaler()
X_train = X_train.copy()
X_test = X_test.copy()
X_train[NUM] = sc.fit_transform(X_train[NUM])   # 학습: 기준을 배우고 변환
X_test[NUM] = sc.transform(X_test[NUM])         # 평가: 배운 기준으로 변환만

print(f"학습 {len(X_train)}건 (양성 {int(y_train.sum())}) / "
      f"평가 {len(X_test)}건 (양성 {int(y_test.sum())})")
X_train.head(3)
```

```
원핫인코딩 후 열 개수: 35
학습 1200건 (양성 60) / 평가 400건 (양성 20)
```

	distance_m	sla_target_min	battery_start_pct	line_stock_at_request_min	shift_Day	shift_Night	requ
1415	1.484	0.716	-0.635	0.595	False	True	
484	-2.020	-0.978	-2.250	-0.433	True	False	
1231	1.380	0.716	0.393	0.573	False	True	

---

12. 모델 3종 학습과 성능평가

모델	방식	특징
로지스틱 회귀	변수에 가중치를 곱해 더한 뒤 확률로 변환	해석이 쉽다
의사결정나무	스무고개처럼 질문을 이어가며 나눔	그림으로 볼 수 있다
랜덤포레스트	나무 300그루의 다수결	성능이 가장 좋은 편

`class_weight="balanced"` **를 반드시 넣으세요.** 양성이 드물면 모델은 "전부 음성"이라고 답하는 것이 손해가 적다고 판단합니다. 이 옵션은 **드문 쪽을 틀렸을 때 벌점을 크게** 주어 그것을 막습니다.

혼동행렬과 지표

	예측: 음성	예측: 양성
실제 음성	TN	**FP (헛경보)**
실제 양성	**FN (놓침)**	TP

- **Recall** = TP/(TP+FN) : 실제 양성 중 몇 %를 잡았나 → **놓치면 안 될 때 중요**
- **Precision** = TP/(TP+FP) : 경보 중 몇 %가 진짜였나 → **헛경보 비용이 클 때 중요**
- **ROC-AUC** : 임계값과 무관하게 "양성에 더 높은 점수를 줄 확률". 0.5=무작위

```python
models = {
    "로지스틱회귀": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "의사결정나무": DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                                          class_weight="balanced", random_state=42),
    "랜덤포레스트": RandomForestClassifier(n_estimators=300, max_depth=8,
                                          min_samples_leaf=5, class_weight="balanced",
                                          random_state=42),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    proba = m.predict_proba(X_test)[:, 1]
    pred = m.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"모델": name, "TP": tp, "FN": fn, "FP": fp, "TN": tn,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0),
                 "ROC_AUC": roc_auc_score(y_test, proba)})
    print(f"===== {name} =====")
    print(classification_report(y_test, pred, zero_division=0))

result = pd.DataFrame(rows).set_index("모델")
display(result.round(3))

base = 1 - y_test.mean()
print(f"참고: '전부 음성' 모델의 정확도 = {base*100:.1f}% (이 숫자는 Recall 0 이다)")
```

```
===== 로지스틱회귀 =====
              precision    recall  f1-score   support

           0       1.00      0.92      0.96       380
           1       0.40      0.95      0.56        20

    accuracy                           0.93       400
   macro avg       0.70      0.94      0.76       400
weighted avg       0.97      0.93      0.94       400

===== 의사결정나무 =====
              precision    recall  f1-score   support

           0       0.99      0.92      0.95       380
           1       0.36      0.90      0.51        20

    accuracy                           0.92       400
   macro avg       0.68      0.91      0.73       400
weighted avg       0.96      0.92      0.93       400

===== 랜덤포레스트 =====
              precision    recall  f1-score   support

           0       0.99      0.96      0.98       380
           1       0.53      0.85      0.65        20

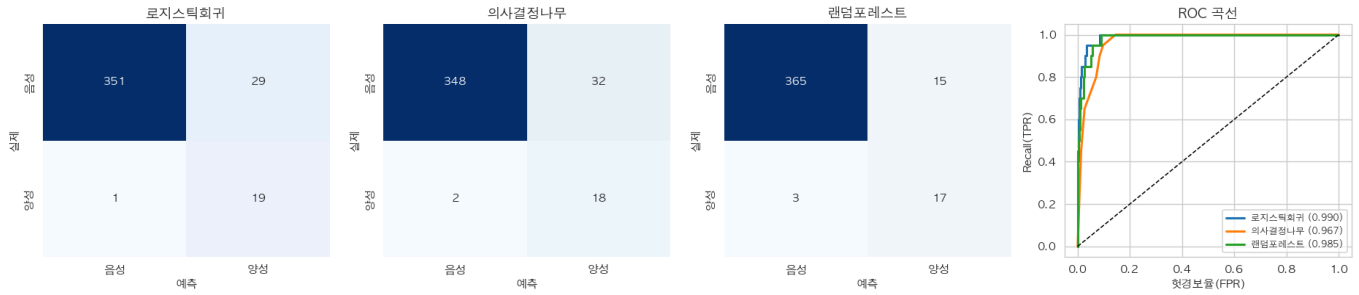
    accuracy                           0.95       400
   macro avg       0.76      0.91      0.81       400
weighted avg       0.97      0.95      0.96       400
```

모델	TP	FN	FP	TN	Recall	Precision	ROC_AUC
로지스틱회귀	19	1	29	351	0.950	0.396	0.990
의사결정나무	18	2	32	348	0.900	0.360	0.967
랜덤포레스트	17	3	15	365	0.850	0.531	0.985

참고: '전부 음성' 모델의 정확도 = 95.0% (이 숫자는 Recall 0 이다)

```
# 혼동행렬 + ROC 곡선
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8))
for ax, (name, m) in zip(axes[:3], models.items()):
    cm = confusion_matrix(y_test, m.predict(X_test), labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["음성", "양성"], yticklabels=["음성", "양성"])
    ax.set_title(name)
    ax.set_xlabel("예측")
    ax.set_ylabel("실제")

for name, m in models.items():
    proba = m.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, proba)
    axes[3].plot(fpr, tpr, lw=2, label=f"{name} ({roc_auc_score(y_test, proba):.3f})")
axes[3].plot([0, 1], [0, 1], "k--", lw=1)
axes[3].set_xlabel("헛경보율(FPR)")
axes[3].set_ylabel("Recall(TPR)")
axes[3].set_title("ROC 곡선")
axes[3].legend(fontsize=8)
plt.tight_layout()
plt.show()
```



---

13. 임계값 조정 — 모델을 바꾸지 않고 운영점 바꾸기

모델은 0/1이 아니라 **확률**을 출력합니다. `predict()` 는 0.5를 기준으로 자르는데, 이 기준은 **바꿀 수 있습니다.**

- 임계값을 **낮추면**: 경보가 늘고 → Recall↑, Precision↓
- 임계값을 **높이면**: 경보가 줄고 → Recall↓, Precision↑

현장 제약을 먼저 정하고 임계값을 고르는 것이 올바른 순서입니다.

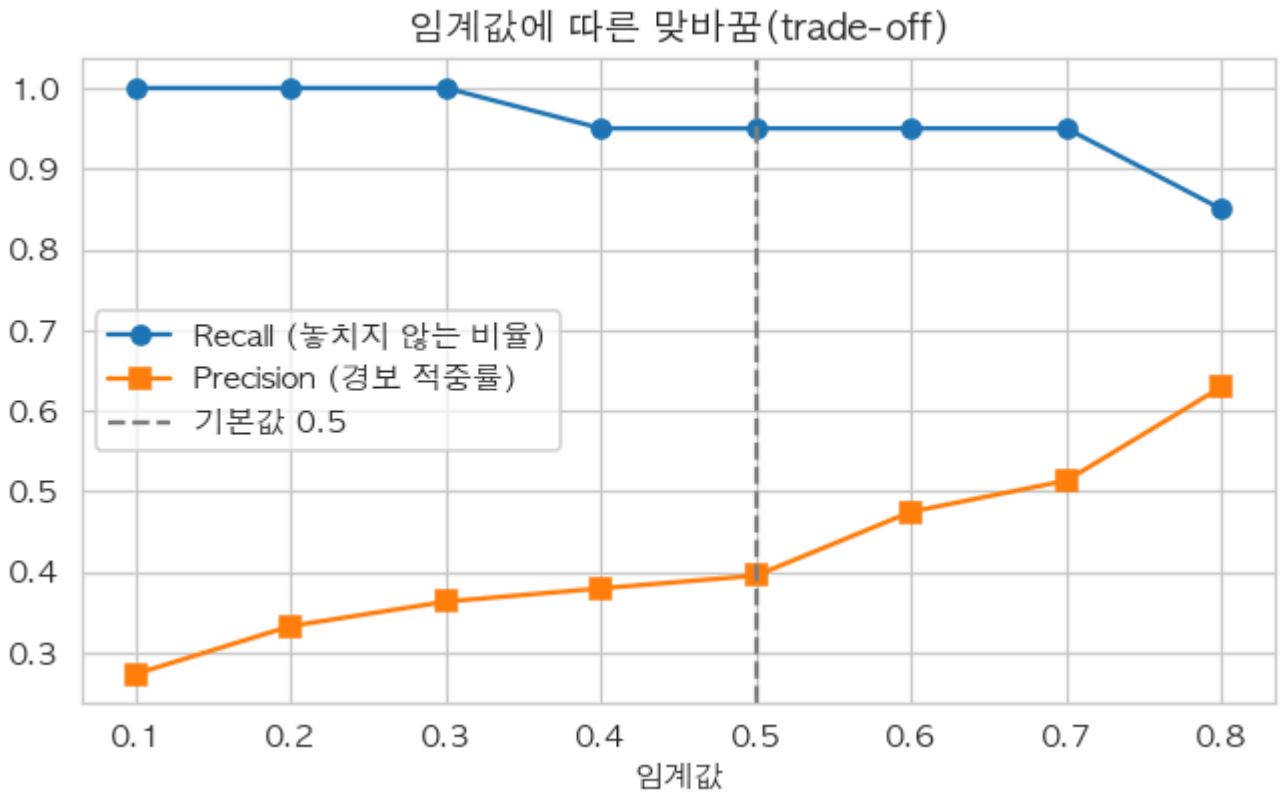
```python
best_name = result["ROC_AUC"].idxmax()          # 임계값과 무관한 지표로 기준 모델 선택
best = models[best_name]
proba = best.predict_proba(X_test)[:, 1]
print("기준 모델:", best_name)

rows = []
for th in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    pred = (proba >= th).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"임계값": th, "경보건수": int(pred.sum()), "TP(잡음)": tp,
                 "FN(놓침)": fn, "FP(헛경보)": fp,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0)})
th_tbl = pd.DataFrame(rows)
display(th_tbl.round(3))

plt.figure(figsize=(7, 3.8))
plt.plot(th_tbl["임계값"], th_tbl["Recall"], "o-", label="Recall (놓치지 않는 비율)")
plt.plot(th_tbl["임계값"], th_tbl["Precision"], "s-", label="Precision (경보 적중률)")
plt.axvline(0.5, color="gray", ls="--", label="기본값 0.5")
plt.xlabel("임계값")
plt.legend()
plt.title("임계값에 따른 맞바꿈(trade-off)")
plt.show()
```

기준 모델: 로지스틱회귀

	임계값	경보건수	TP(잡음)	FN(놓침)	FP(헛경보)	Recall	Precision
0	0.100	73	20	0	53	1.000	0.274
1	0.200	60	20	0	40	1.000	0.333
2	0.300	55	20	0	35	1.000	0.364
3	0.400	50	19	1	31	0.950	0.380
4	0.500	48	19	1	29	0.950	0.396
5	0.600	40	19	1	21	0.950	0.475
6	0.700	37	19	1	18	0.950	0.514
7	0.800	27	17	3	10	0.850	0.630



---

14. 특성중요도 — 모델은 무엇을 보고 판단했나

랜덤포레스트는 "이 변수로 나눴을 때 양성/음성이 얼마나 깔끔하게 갈리는가"를 기준으로 분기합니다. 그 기여도를 모두 합한 것이 **특성중요도**입니다(합 = 1).

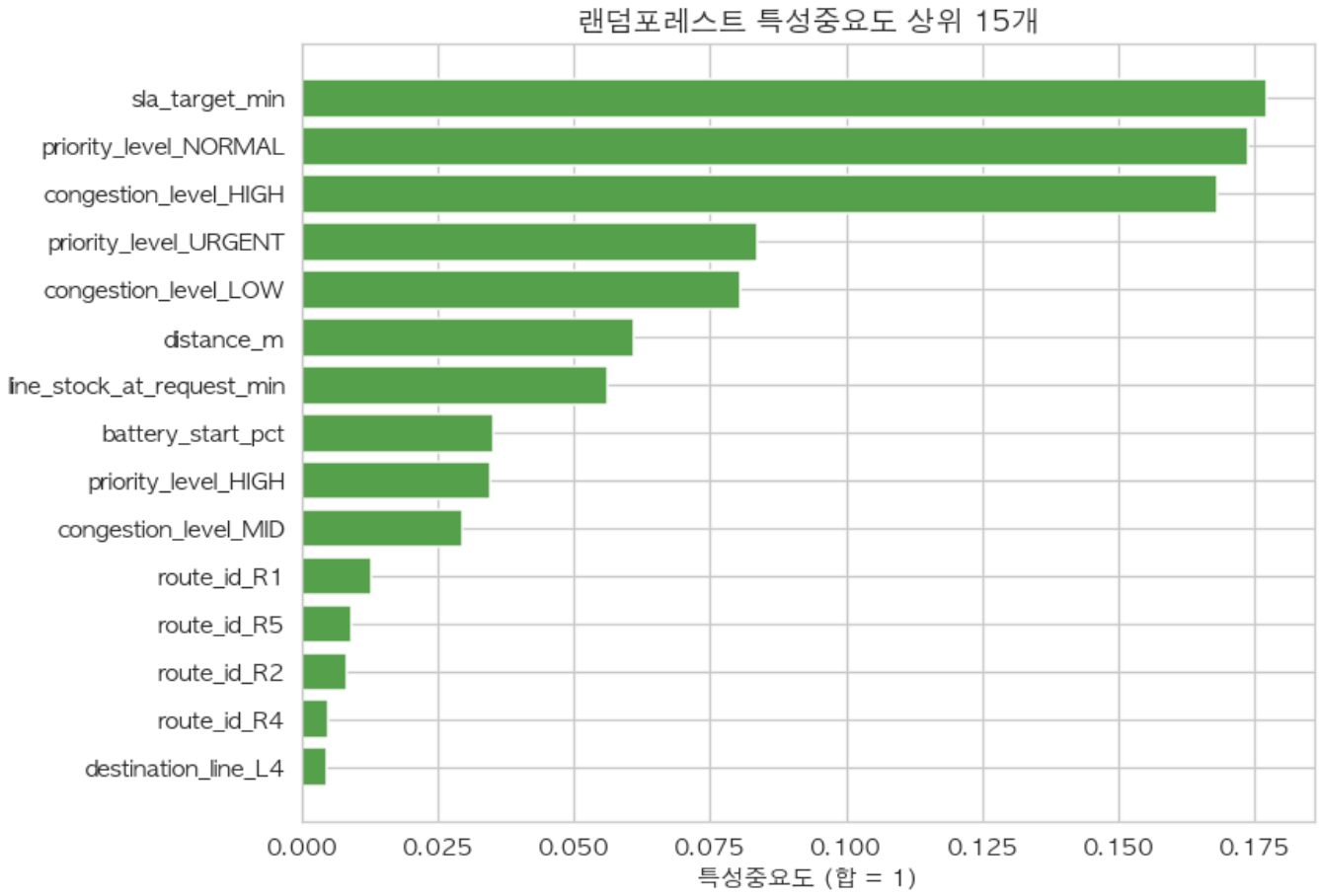
> **주의**: 특성중요도는 **"모델이 많이 참고한 변수"** 이지 **"현실의 원인"** 이 아닙니다. 인과를 주장하려면 현장에서 조건을 바꿔 보는 확인이 필요합니다.

```
rf = models["랜덤포레스트"]
imp = pd.DataFrame({"변수": X_train.columns,
                    "중요도": rf.feature_importances_}).sort_values("중요도", ascending=False)
display(imp.head(10).round(4))

top = imp.head(15)[::-1]
plt.figure(figsize=(8, 5.5))
plt.barh(top["변수"], top["중요도"], color="#59a14f")
plt.xlabel("특성중요도 (합 = 1)")
plt.title("랜덤포레스트 특성중요도 상위 15개")
plt.tight_layout()
plt.show()

print("상위 5개 변수가 전체 중요도의", f"{imp.head(5)['중요도'].sum()*100:.0f}% 를 차지합니다.")
```

	변수	중요도
1	sla_target_min	0.177
19	priority_level_NORMAL	0.174
32	congestion_level_HIGH	0.168
20	priority_level_URGENT	0.084
33	congestion_level_LOW	0.080
0	distance_m	0.061
3	line_stock_at_request_min	0.056
2	battery_start_pct	0.035
18	priority_level_HIGH	0.035
34	congestion_level_MID	0.029



상위 5개 변수가 전체 중요도의 68% 를 차지합니다.

---

15. 정리

이 데이터에서 확인한 것

- SLA 미준수는 80건(5.0%)뿐이다. **정확도 95%는 아무것도 안 해도 나온다.**
- **NORMAL 요청 미준수 0건, URGENT 28.5%** → 미준수는 목표시간이 짧은 요청에 집중된다.
- 미준수 요청의 평균 이동거리 415m vs 준수 339m. 평균 배차대기 126초 vs 82초. 평균 이동시간 407초 vs 238초 → **거리·배차대기·이동시간 모두 차이가 있다.**
- 혼잡도 HIGH 312건 중 62건(19.9%)이 미준수 → **혼잡이 가장 강한 단일 위험요인**이다.
- 경로 R5·R4에서 미준수가 상대적으로 많다 → 경로 자체를 재검토할 근거.
- SLA 미준수 80건 중 **라인정지는 25건**. 나머지는 라인 재고가 버텨 줬다 → 진짜 위험은 **"재고 적음 + 배송 지연"의 조합**이다.
- `sla_target_min`은 `priority_level`과 1:1 대응(8/12/18분) → 중복 정보.
- 결측: 출발 배터리 10건, 이동시간 14건. 충전 66건은 배터리 증가가 **정상**이다.

결론과 다음 단계

- NORMAL 요청은 SLA 위반이 0건이라, 모델이 "NORMAL이면 안전"을 먼저 배운다. 성능이 높게 나오는 것은 **문제가 쉬워서**다.
- 진짜 과제는 **URGENT 안에서의 구분**(165건 중 47건 위반)과 **라인정지로 이어질 요청 찾기**(위반 80건 중 정지 25건)다.
- 다음 단계: `line_stock_at_request_min`(요청 당시 잔여재고)과 결합한 "재고가 빠듯한데 늦을 것 같은 요청" 플래그를 만들어 보라.

공통으로 기억할 5가지

#	원칙
1	**정확도만 보지 않는다** — 양성이 드물면 아무것도 안 해도 95%가 넘는다
2	**놓침(FN)과 헛경보(FP)의 비용은 다르다** — 어느 쪽이 비싼지는 현장이 정한다
3	**성능이 너무 좋으면 누출을 의심한다** — 결과 변수를 넣지 않았는지 확인
4	**임계값은 운영 정책이다** — 모델을 바꾸지 않고도 경보 수를 조절할 수 있다
5	**상관은 인과가 아니다** — 특성중요도는 원인이 아니라 단서다

연습문제 / 생각해 볼 질문

아래 문항은 정답을 맞히는 시험이 아니라, **발표자료에 쓸 문장을 만드는 연습**입니다. 코드를 조금씩 바꿔 보고, 결과를 한 문장으로 정리해 보세요.

1. 우선순위별 실제 배송시간 분포(박스플롯)에 각 SLA 목표선(8/12/18분)을 그려 보세요. URGENT의 목표가 달성 가능한 수준입니까?
2. 혼잡도 × 경로 교차표로 미준수율을 구하고, '혼잡 때문인지 경로 때문인지' 구분해 보세요.
3. 세트 A(요청접수 시점)와 세트 B(배차 이후) 모델의 Recall을 비교하세요. 조치 가능성까지 고려하면 어느 모델을 쓰겠습니까?
4. `actual_delivery_min`을 특성에 넣어 AUC가 1.0이 되는 것을 확인하고 누출을 설명하세요.